Sean Yan
64322415
CPSC 320 A2

1 Slow as a logarithm

expanding on $\log_b(n) \in O(n^x)$ with the definition of big O,

$$\exists c \in \mathbb{R}, \exists n_0 \in \mathbb{R}, c > 0$$
$$\forall n \geq n_0 > 0,$$
$$\log_b(n) \leq c \cdot n^x$$

for every $b > 1$ and $x > 0$
let $n_0 = 1$ and $c = 1$

$$\log_b(1) \leq 1 \cdot 1^x$$
$$0 \leq 1 \cdot 1^x$$
$$0 \leq 1$$

so $n_0 = 1$ and $c = 1$ is an valid choosing of c and n_0

and we want, by the definition of little o

$$\lim_{n \to \infty} \left(\frac{\log_b(n)}{n^x} \right) = 0$$

Question 2: I wanna be the very best

a)

this is a combination question.

Question 3

3b

base case $n = 1$

$$1^1 \geq (1+1)^{1-1}$$
$$1 \geq 2^0$$
$$1 \geq 1$$

base case proven

Assume for induction hypothesis

$$m^m \geq (m+1)^{m-1}$$

for $2 \leq m \leq n$

and we want to prove $n+1$

$$(n+1)^{n+1} \geq (n+1+1)^{n+1-1}$$
$$(n+1)^{n+1} \geq (n+2)^n$$
$$(n+1)^n(n+1) \geq (n+2)^n$$
$$(n+1)^n \geq \left(\frac{(n+2)^n}{n+1}\right)$$
$$\frac{(n+1)^n}{(n+2)^n} \geq \frac{1}{n+1}$$
$$\left(\frac{n+1}{n+2}\right)^n \geq \frac{1}{n+1}$$

from the inductive hypothesis

$$n^n \geq (n+1)^{n-1}$$
$$n^n \geq \frac{(n+1)^n}{n+1}$$
$$\left(\frac{n}{n+1}\right)^n \geq \frac{1}{n+1}$$

Question 4

`H` is home `A` is school A `B` is school B `W` is work

a)

3 times for symmetric

6 times for asymmetric

b)

2 times for symmetric, because running from (home, A) gives implicitly (home, B), and we just need to run it again from (A,B)

3 times for asymmetric.

we need all 6,

```
(H, A)
(A, H)
(A, B)
(B, A)
(H, B)
(B, H)
```

running noodle maps on `(H, A)` gives also `(H, B)`

again, on `(A, B)` gives `(A, H)`

again on `(B, H)` gives `(B, A)`

thus, getting all 6 in 3 runs

c)

if We are working only under part a with part c,

5 times if symmetric.

```
(H, A)
(H, B)
(A, B)
(A, W)
(B, W)
```

if asymmetric, it's 6 times, one per each edge

```
(H, A)
(H, B)
(A, B)
(B, A)
(A, W)
(B, W)
```

now working under part b

2 times for symmetric. in this case we need four edges,

```
(H, A)
(B, A)
(B, W)
(A, W)
```

run it on `(B, A)` gives `(B, W)`

run it again on `(A, W)` gives `(H, A)` as well.

in the asymmetric case, it's 3 times

```
(H, A)
(H, B)

(A, W)
(A, B)

(B, W)
(B, A)
```

run it on `(A, W)` gives `(A, B)`

run it on `(B, A)` gives, `(B, W)`

run it on `(H, A)` gives `(H, B)`